UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
ESCUELA DE POSGRADO



Proyecto del Curso

SISTEMA INTEGRADO DE PROCESAMIENTO EN STREAMING PARA COMERCIO ELECTRÓNICO

AUTOR(ES)
Jesús Ricardo Delgado Alvarado
Guillermo Diego Calderón Canchanya
Mentor Alexis Naquiche Ventura
Erik Rogers Poma García

DOCENTE
Heider Sánchez Enríquez

CURSO
Estructura de Datos y Algoritmos

Lima – Perú
2026

1 Resumen y contexto del proyecto

Se implementó un pipeline educativo de diez etapas para procesar actividad histórica de comercio electrónico como un stream finito de microlotes. El sistema lee todo el archivo seleccionado, conserva estado entre lotes y utiliza sus estructuras para producir recomendaciones explicadas, alertas de revisión, consultas por precio y rankings de compras aceptadas.

La ejecución integrada guardada recibió 1.000 eventos originales y ocho eventos simulados separados. Procesó los 1.008 registros en tres lotes, sin rechazos ni duplicados, y produjo dos alertas de revisión. Esto demuestra el funcionamiento del caso local; no prueba detección de fraude real ni rendimiento de producción.

Problema de negocio y alcance

Los eventos de vistas, carrito y compras permiten estudiar acceso frecuente a datos, prioridad de atención, similitud de productos e influencia en un grafo. El objetivo es conectar esas operaciones mediante estructuras apropiadas y evaluar sus costos frente a referencias exactas o más simples. El histórico no llega desde una plataforma de mensajería: se reproduce desde Parquet en bloques.

El visor web ejecuta el mismo motor de main.py mediante observadores explícitos. Muestra el lote y la etapa activa, permite detener la ejecución y conserva los resultados al terminar. Sus partículas y esquemas internos son recursos didácticos; los tiempos del visor y sus pausas no constituyen un benchmark.

Datos originales y escenario controlado

Los CSV de octubre y noviembre de 2019 contienen actividad comercial, identificadores, precio, categoría, marca, usuario y sesión. No incluyen order_id ni etiquetas verificadas de fraude. La preparación incremental conserva el tipo de evento y genera identificadores SHA-256 estables. Los IDs antiguos truncados se migran al leer, conservando event_id_original y sin reescribir los archivos.

Los ocho registros adicionales están marcados con is_simulated=true y representan órdenes explícitas y actividad repetida de una cuenta. Se pueden desactivar con --sin-escenario. Las compras originales sin orden se contabilizan, pero no se convierten automáticamente en pares de co-compra. Los grafos del benchmark usan otra simulación reproducible, descrita en la metodología.

Guía de lectura

Las secciones 2 y 3 describen el flujo y las estructuras base; 4 a 6 desarrollan las cinco familias avanzadas; 7 a 9 presentan metodología y resultados; 10 documenta ejecución y validación; 11 reúne conclusiones y referencias. Las cifras experimentales corresponden a los JSON guardados el 29 de octubre de 2026.

2 Arquitectura y flujo de datos

Cada lote atraviesa las diez etapas. La salida de una etapa aporta registros, candidatos o decisiones que se consumen después. El estado persiste entre lotes: los eventos no desaparecen de todas las estructuras cuando una cola se vacía.



Figura 1. Arquitectura implementada. Las flechas representan dependencias de datos de una ejecución secuencial por lote.

Paso	Información que se entrega
0 → 1	Lote completo extraído del búfer FIFO.
1 → 2	Eventos válidos, normalizados, únicos y ordenados por fecha.
2 → 3	Eventos atendidos por prioridad, fecha y orden estable de llegada.
3 → 4	Registros recuperables que alimentan catálogo e índice de precios.
4 → 5 → 6	Candidatos por rango y vecinos exactos; después se verifica el riesgo.
6 → 7 → 8	Eventos aceptados y candidatos similares que enriquecen co-compra.
8 → 9	PageRank, comunidades, complementos y candidatos para la salida.

Las colas FIFO y heap se vacían al atenderlas. Los eventos y el historial exacto se guardan en SQLite; el B+ conserva sus propias páginas y metadatos en disco. Catálogo, vectores y grafo permanecen en RAM. Leer por bloques limita el lector, pero no hace constante la memoria del sistema completo.

3 Estructuras base y decisiones de diseño

Etapas 0 y 1 Ingesta y preprocesamiento

FIFO usa collections.deque con capacidad del lote: insertar y extraer cuestan O(1), con espacio O(b). Es un búfer local y secuencial, sin productores concurrentes ni garantía de absorción ilimitada de tráfico. Evita el desplazamiento de elementos de list.pop(0).

El preprocesador valida columnas, fechas UTC y precios finitos no negativos; precio cero es válido. Deduplica dentro del lote y contra historial exacto en disco. Timsort ordena O(b log b). La búsqueda binaria opera sobre un subconjunto histórico ordenado acotado; reconstruirlo y escribir el historial añade costo. No se atribuye esta etapa a una vectorización de pandas ni a una aceleración no medida.

Etapa 2 Prioridad estable

El MinHeap utiliza (prioridad, fecha, contador, evento). Atiende alertas de pago antes de compras y eventos menos urgentes; el contador resuelve empates sin comparar diccionarios. Cada inserción o extracción cuesta O(log b) y vaciar el lote O(b log b). Su ventaja es expresar atención incremental por prioridad, no mejorar asintóticamente la ordenación completa del lote.

Etapa 3 Almacenamiento y caché

Un hash reciente y una caché LRU con OrderedDict reducen accesos repetidos. Sus operaciones en RAM tienen costo esperado O(1), con capacidades explícitas. SQLite conserva los eventos completos; sus accesos no se describen como O(1). Reemplazar un registro invalida la copia cacheada. Se distingue un diccionario vacío válido de la ausencia de datos.

Etapa 4 Índice B+ persistente

El árbol propio decide separadores, divisiones y recorrido de hojas. SQLite almacena páginas por ID y metadatos; no sustituye el algoritmo del B+. La clave compuesta (precio, product_id) permite consultar rangos inclusivos sin confundir productos con igual precio. La búsqueda requiere O(log_B n) páginas y el rango añade las páginas y resultados recorridos. Una caché de páginas limita su residencia en RAM.

El catálogo conserva el primer precio observado como referencia inmutable durante la ejecución. No representa stock ni precio actual. Las pruebas validan invariantes, hojas enlazadas, búsqueda, rangos y reapertura. No se implementa borrado del B+.

Etapas 5 y 9 Vecinos y presentación

KD-Tree particiona vectores de producto y poda ramas según distancia. La construcción por ordenación recursiva cuesta O(n log² n); una consulta puede recorrer todos los puntos. Se contrasta con búsqueda exhaustiva y se excluye el propio producto. El Trie devuelve coincidencias de prefijo, con costo dependiente también de las salidas. Top-K cuenta compras aceptadas y cuesta aproximadamente O(p log k) para p productos.

4 Filtros probabilísticos y similitud de conjuntos

Familia 1 Bloom Filter y Count-Min Sketch

Bloom [1] usa m = ceil(−n ln(p)/(ln 2)²) bits y aproximadamente k = (m/n) ln 2 hashes. La implementación empaqueta bits en bytearray. Insertar o consultar cuesta O(k); un positivo puede ser falso y un negativo descarta pertenencia para elementos insertados en esa instancia. El set exacto es su referencia experimental.

Count-Min Sketch [2] mantiene una matriz uint64 y estima una frecuencia con el mínimo de sus filas. Actualizar y consultar cuestan O(d), con espacio O(d·w). Las colisiones pueden sobreestimar. Se comprueban desbordamientos y se prohíben restas. El control es Counter, registrando error absoluto y sobreestimación máxima.

En el pipeline, la clave es la cuenta, la ventana es de 300 segundos y el umbral es tres intentos. Bloom recuerda cuentas señaladas y un conjunto exacto confirma pertenencia; CMS estima frecuencia y un contador exacto confirma la decisión. Los eventos que requieren revisión no alimentan la analítica posterior. Las acciones previas al umbral no se revierten.

La ventana avanza con la mayor fecha observada. Un heap temporal permite caducar registros aunque la prioridad altere su orden de atención; los demasiado tardíos se identifican. Al caducar eventos se reconstruye CMS. Esa verificación consume memoria y tiempo adicionales: no corresponde atribuir O(1) ni espacio sublineal a toda la política de riesgo.

Resultado 1K: tasa de falsos positivos Bloom = 0.600%, sin falsos negativos en la prueba; error absoluto medio CMS = 0.002976, sobreestimación máxima = 1. CMS usó cinco filas y ancho 2.000. Son pruebas de estructuras, no métricas de fraude real.

Familia 2 MinHash y LSH

MinHash [3] aproxima Jaccard mediante firmas de 64 mínimos hash. LSH divide cada firma en 16 bandas de cuatro filas y recupera candidatos desde buckets con bandas coincidentes. La probabilidad idealizada de coincidencia para similitud s es $1-(1-s^r)^b$. Crear una firma cuesta O(k·|A|); consultar depende de los candidatos y de los buckets, sin garantía O(1) total.

Las canastas operativas agrupan compras por usuario, con retención de hasta 10.000 cuentas activas. No se presentan como órdenes. El experimento forma conjuntos de interacción por sesión, compara con Jaccard exacto para las mismas consultas y umbral 0,3, y mide recall, precisión y error de firma. La actualización y retirada de canastas también modifican sus buckets.

En 307 conjuntos y cinco consultas, recall y precisión fueron 1,000 en las 100 repeticiones. Es una muestra pequeña y no demuestra calidad universal. Se investigan RedisBloom y Guava como referencias de filtrado, y datasketch y Spark MinHashLSH para similitud; no son dependencias del algoritmo implementado.

5 Búsqueda vectorial aproximada

Familia 3 HNSW frente a KNN exacto

HNSW [4] organiza productos en un grafo de proximidad con niveles aleatorios. La consulta desciende de forma voraz por capas superiores y explora una frontera de candidatos en la inferior. Una selección de vecinos diversos evita concentrar enlaces en grupos muy cercanos. La búsqueda explora candidatos sobre ese grafo y evita un recorrido exhaustivo del catálogo como estrategia de consulta.

Se usan M=8, ef_construction=48, ef_search=32 y semilla 42. M limita vecinos; ef controla amplitud de exploración. Aumentar ef puede mejorar recall a costa de trabajo. El costo depende del corpus, la dimensión y los parámetros: no se garantiza O(log n) para toda consulta ni recall perfecto. Los vectores y enlaces permanecen en RAM.

Representación de producto y referencia justa

Los vectores tienen 17 atributos reproducibles: precio logarítmico normalizado y buckets estables de categoría y marca. No son embeddings semánticos aprendidos; el hashing puede producir colisiones y empates. KD-Tree y la referencia exhaustiva usan las mismas coordenadas y distancia. En el benchmark de HNSW se conserva el mismo corpus, las mismas cinco consultas, k=5 y exclusión del propio ID.

Se informa recall@5 convencional por identificador y otra métrica que acepta distancias empatadas. La segunda ayuda a explicar empates y no reemplaza silenciosamente a la convencional. En el experimento 1K, ambas fueron 1,000 ± 0,000 para 672 productos. Repetir la misma semilla no amplía la diversidad de consultas ni demuestra robustez ante otras distribuciones.

Componente medido	Media ± desviación en ms
Construcción HNSW	1.423,186 ± 40,834
Cinco consultas HNSW	3,706 ± 0,433
Cinco consultas KNN exacto	19,219 ± 0,712
Carga total HNSW con comprobaciones	1.427,325 ± 40,889

La consulta HNSW fue más rápida para esta carga, pero construir su índice fue mucho más costoso que resolver las cinco consultas exactas. La elección depende de cuántas consultas reutilicen el índice, del recall requerido y de la memoria disponible. No se suman construcción y consulta para declarar una ventaja de latencia por petición.

Uso en la salida y límites

La etapa 7 aporta candidatos ANN y canastas similares. El servicio combina esas señales con vecinos exactos KD, rango de precios, co-compra y PageRank, mostrando el origen del puntaje. Los pesos son demostrativos, sin validación de conversión comercial. Hnswlib y FAISS son referencias de implementación; el backend del proyecto es un HNSW educativo propio, sin todas sus optimizaciones.

6 Influencia y comunidades de productos

Modelo de grafo y procedencia

El grafo operativo es producto-producto, no usuario-producto. Una orden explícita enlaza cada pareja de artículos una sola vez; nuevas órdenes aumentan el peso. La membresía de órdenes se conserva en SQLite para reconocer datos de lotes posteriores. Las compras sin order_id se contabilizan aparte. El grafo y sus índices analíticos permanecen en RAM.

El caso de demostración incorpora dos órdenes controladas. Para los experimentos se crean órdenes SIMULADAS agrupando de tres en tres product_id consecutivos del archivo. Esto permite repetir una carga, pero no reconstruye las órdenes reales ni valida patrones comerciales del histórico. El grafo 1K tiene 672 nodos y 997 aristas; ambos algoritmos comparados reciben ese mismo grafo.

Familia 4 PageRank

PageRank [5] propaga influencia con amortiguación 0,85 y pesos de co-compra, usando NetworkX. Su costo depende de la convergencia, aproximadamente O(I·(V+E)) para I iteraciones, con espacio O(V+E). El grado ponderado es la alternativa simple local. El benchmark mide tiempos y solapamiento del top 20; el acuerdo fue 0,900. Ese acuerdo no es precisión de ventas ni NDCG: no existen etiquetas de relevancia comercial para evaluarlas.

Familia 5 Louvain y propagación de etiquetas

Louvain [6] mueve nodos entre comunidades y agrega grupos para mejorar modularidad. Se utiliza nx.community.louvain_communities para obtener particiones mediante optimización de modularidad. Se compara con propagación asíncrona de etiquetas [7]. El costo depende de pasadas, resolución y topología; no se ofrece una cota logarítmica universal.

Método	Modularidad media ± desv.	Estabilidad ARI media ± desv.
Louvain	0,868 ± 0,001	0,723 ± 0,064
Propagación de etiquetas	0,673 ± 0,008	0,562 ± 0,051

Louvain produjo 69,25 ± 0,89 comunidades y propagación de etiquetas 182,63 ± 4,53. El JSON conserva sus tamaños por corrida. La semilla cambia a 42+repetición para estudiar estabilidad.

ARI compara cada partición contra la primera corrida, que tiene ARI=1 por definición. No es una validación contra comunidades verdaderas. Louvain alcanzó mayor modularidad y estabilidad promedio en este grafo simulado; la conclusión no se extiende a todos los grafos ni demuestra segmentos reales de clientes.

NetworkX es la implementación usada en estas dos familias. GraphX, Neo4j GDS, igraph y cuGraph son referencias para investigar otras escalas y plataformas, no servicios conectados a esta versión.

7 Metodología experimental

El benchmark se ejecuta fuera del visor. Cada repetición reconstruye las estructuras, mantiene su objeto hasta medir memoria y conserva datos crudos. La preparación de entradas y las referencias previas se excluyen de la medición. Los tiempos totales incluyen la carga de construcción, consultas y comprobaciones definidas por experimento; no equivalen automáticamente a latencia por consulta.

Campo	Configuración registrada
Plataforma	Windows-11-10.0.26200-SP0
Python	3.14.4
Semilla base	42
Escala estadística	1K con 100 repeticiones y 19 algoritmos o controles
Población 1K	1.000 eventos; 672 claves/vector-producto; 307 conjuntos
Consultas y dimensiones	5 consultas; 17 dimensiones; k=5 en ANN
Grafo 1K	672 nodos; 997 aristas; órdenes simuladas
Recorte configurado 1K	1.000 entidades; sin recorte efectivo en este archivo

Comparaciones por familia de algoritmos

La evaluación compara cada familia con una referencia exacta o una alternativa simple: Bloom frente a set; CMS frente a Counter; MinHash+LSH frente a Jaccard; HNSW frente a KNN exacto; PageRank frente a grado; y Louvain frente a propagación de etiquetas. También se evalúan las estructuras base, incluido el B+ en disco. Cada comparación utiliza la misma población y las mismas consultas dentro de su familia.

Se publican media, desviación estándar muestral, memoria actual y pico, calidad y poblaciones reales. Construcción y consulta ANN se separan. Eventos, vectores, conjuntos y nodos no son la misma unidad: el tamaño del Parquet no demuestra que un índice contenga esa cantidad de productos distintos.

Memoria y reproducibilidad

tracemalloc registra asignaciones Python actuales y pico de la región medida. RSS antes y después describe el proceso; no es el pico nativo completo ni incluye como memoria del algoritmo todos los insumos preparados antes. Las tablas usan KiB=1.024 bytes. No se presentan estas cifras como RAM total ni como tamaño de la base en disco.

Los JSON registran versiones, plataforma, fecha, semilla, parámetros, procedencia del grafo, límites de entidades y huellas SHA-256 del código. Las 100 repeticiones caracterizan variación de ejecución sobre una carga fija, no 100 muestras independientes de clientes. Una desviación cero de calidad puede reflejar entradas y semillas constantes.

Dependencias registradas: numpy 2.5.4; pandas 3.1.0; pyarrow 26.0.0; networkx 3.7; scikit-learn 1.9.1; mmh3 5.3.1; psutil 7.2.2.

8 Resultados medidos en la escala 1K

Se obtuvieron 1.900 registros de medición: 19 algoritmos o controles por 100 repeticiones. Cada fila resume su propia carga de trabajo y población. Las diferencias entre filas no prueban por sí solas que una estructura pueda sustituir a otra.

Algoritmo o control	Tiempo total ms media ± desv.	Actual KiB	Pico KiB
FIFO	0,180 ± 0,023	9,1	9,1
Preprocesamiento	3,553 ± 0,216	25,3	87,4
Heap	2,740 ± 3,209	5,8	204,1
Hash y LRU	0,852 ± 0,111	274,2	275,1
B+ en disco	153,439 ± 5,470	63,1	66,3
KD-Tree	13,902 ± 0,917	93,2	100,0
Bloom	7,142 ± 0,221	1,2	1,7
Set exacto	0,519 ± 0,055	48,0	48,3
Count-Min Sketch	16,100 ± 0,886	78,3	95,4
Counter	0,480 ± 0,066	33,9	35,2
MinHash y LSH	55,333 ± 4,952	2.364,6	2.368,6
Jaccard exacto	6,167 ± 0,300	2,3	21,0
HNSW	1.427,325 ± 40,889	783,0	802,4
KNN exacto	19,221 ± 0,712	5,5	41,9
PageRank	5,850 ± 0,319	7,3	270,5
Grado ponderado	1,650 ± 0,134	0,2	28,6
Louvain	93,609 ± 7,025	20,3	856,9
Propagación etiquetas	43,846 ± 4,980	10,8	132,0
Trie y Top-K	2,903 ± 0,331	165,1	248,3

La memoria representa la media de asignaciones actuales y de picos Python por repetición. El tiempo total comprende construcción y la carga experimental correspondiente. En HNSW, las consultas tardaron 3,706 ± 0,433 ms y la construcción 1.423,186 ± 40,834 ms; el detalle evita confundir el costo del índice con el de una petición.

En esta carga pequeña, los controles nativos set y Counter fueron más rápidos que Bloom y CMS. MinHash+LSH tuvo mayor costo total que Jaccard exacto. Es consistente con el costo de construir índices y ejecutar hashes en Python; no invalida los algoritmos ni demuestra que convengan a toda escala. La calidad y el costo deben analizarse juntos.

9 Evaluación exploratoria de escalas mayores

Se completó una pasada funcional por archivo en 10K, 100K, 1M y 10M. Bloom, set, CMS y Counter recibieron las claves de todos los eventos. La analítica se recortó a 300 vectores y 300 conjuntos; los grafos se construyeron con 900 eventos. Se exploraron 5.000 filas para recoger esas entidades. El análisis exploratorio caracteriza el comportamiento observado en una ejecución por escala. Sus resultados se interpretan sin estimar variabilidad entre repeticiones.

Eventos	Claves distintas	Repet.	Vectores / conjuntos	Nodos / aristas
10.000	4.270	1	300 / 300	621 / 898
100.000	20.675	1	300 / 300	621 / 898
1.000.000	66.001	1	300 / 300	621 / 898
10.000.000	133.917	1	300 / 300	621 / 898

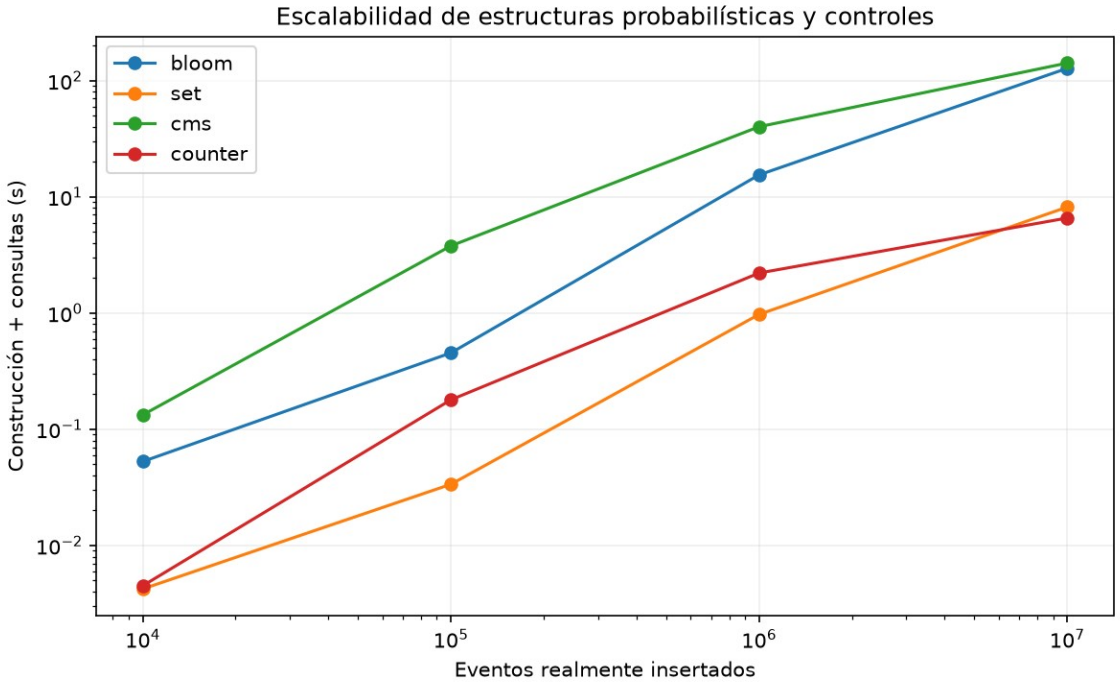


Figura 2. Tiempos totales de la evaluación exploratoria de filtrado y conteo. Ejes log-log; una repetición por escala, sin intervalos de variación. La figura no representa escalabilidad de HNSW, LSH o comunidades a 10M entidades.

Las curvas de métodos con corpus analíticos constantes no permiten inferir su crecimiento con el total de eventos. En esta evaluación, la variación por tamaño de archivo se analiza para las estructuras de filtrado y conteo que reciben todos los eventos. La similitud y los grafos conservan los límites declarados de población. Una sola repetición por escala no permite calcular una desviación estándar representativa ni establecer conclusiones estadísticas de rendimiento.

10 Ejecución integrada y validación

Resultado de negocio guardado

Indicador	Resultado
Eventos originales / simulados	1000 / 8
Eventos procesados / lotes	1008 / 3
Rechazos / duplicados	0 / 0
Alertas de revisión	2
Compras aceptadas / sin orden explícita	12 / 6

La salida incluye recomendaciones con puntajes y señales, alertas, Top-K y candidatos de precio. El catálogo y el grafo se reutilizan entre lotes. Los datos persistidos de una corrida se separan en data/runtime con un identificador único; ejecutar otra corrida no fusiona silenciosamente ambas. La deduplicación histórica opera dentro de su estado de ejecución.

Pruebas automatizadas

Las 11 pruebas pasan. Cubren inserciones aleatorias e invariantes del B+, rangos y reapertura; actualización y desalojo de LRU; validación y deduplicación entre lotes; prioridad estable; KD frente a exacto; Bloom/CMS, caducidad y confirmación exacta; actualización y retirada LSH; recall de HNSW; co-compra e integración; preparación incremental; pausa, detención y reinicio por HTTP.

Cómo reproducir

Iniciar motor y visor: .\.venv\Scripts\python.exe main.py

Sin visor: .\.venv\Scripts\python.exe main.py --sin-visor

Pruebas: .\.venv\Scripts\python.exe -m unittest discover -s tests -v

Benchmark 1K: .\.venv\Scripts\python.exe benchmarks.py --sizes 1K --repetitions 100 --max-entities 1000 --queries 5

La instalación y preparación CSV están explicadas en README.md. El notebook 01 muestra etapas y consultas; el 02 reutiliza los JSON y permite habilitar nuevas corridas. El visor abre en http://127.0.0.1:8765. La pausa visual se aplica fuera del tiempo observado del algoritmo y Detener coopera con el motor; no fuerza la interrupción de una operación en cualquier instrucción.

Límites operativos

El lector está acotado por lote, pero catálogo, vectores, grafo y algunas verificaciones crecen con entidades distintas. El servidor local mantiene una ejecución compartida, sin autenticación ni aislamiento por usuario. No se presenta como despliegue distribuido listo para producción. Las animaciones no acreditan paralelismo ni rendimiento en tiempo real de una plataforma empresarial.

11 Conclusiones y referencias

El sistema conecta diez etapas mediante salidas que se consumen posteriormente, procesa todos los lotes y mantiene persistencia verificable. Integra un B+ propio en disco, búsqueda KD con poda, LSH por bandas, HNSW multicapa, una ventana temporal con confirmación exacta y detección de comunidades mediante Louvain. La arquitectura distingue el estado transitorio, las estructuras en memoria y los datos conservados en disco.

La evaluación de 1K con 100 repeticiones permite describir la variación temporal de la carga estudiada. Los resultados muestran que el costo de construir índices debe distinguirse de la consulta y que el intercambio entre memoria y calidad depende del algoritmo y de la población. La evaluación exploratoria hasta 10M amplía la observación de filtrado y conteo, mientras mantiene poblaciones acotadas para similitud y grafos. Sus resultados no se extrapolan a latencias ni mejoras universales.

La ausencia de órdenes reales y de etiquetas verificadas de relevancia o fraude limita la validación comercial. Como líneas de trabajo futuro se plantean ampliar las repeticiones y poblaciones analíticas en las escalas mayores, evaluar sensibilidad a parámetros y utilizar datos con relevancia conocida. El proyecto aporta una implementación académica reproducible para estudiar estructuras de datos, aproximación y análisis de grafos en un flujo integrado.

Referencias de algoritmos

[1] Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, 13(7), 422 –426. DOI 10.1145/362686.362692.

[2] Cormode, G., y Muthukrishnan, S. (2005). An improved data stream summary: the count-min sketch and its applications. Journal of Algorithms, 55(1), 58–75. DOI 10.1016/j.jalgor.2003.12.001.

[3] Broder, A. Z. (1997). On the resemblance and containment of documents. SEQUENCES 1997, 21–29. DOI 10.1109/SEQUEN.1997.666900.

[4] Malkov, Y. A., y Yashunin, D. A. (2020; versión previa 2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE TPAMI, 42(4), 824–836. DOI 10.1109/TPAMI.2018.2889473.

[5] Brin, S., y Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. Computer Networks and ISDN Systems, 30(1–7), 107–117. DOI 10.1016/S0169-7552(98)00110-X.

[6] Blondel, V. D., Guillaume, J.-L., Lambiotte, R., y Lefebvre, E. (2008). Fast unfolding of communities in large networks. JSTAT, P10008. DOI 10.1088/1742-5468/2008/10/P10008.

[7] Raghavan, U. N., Albert, R., y Kumara, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. Physical Review E, 76, 036106. DOI 10.1103/PhysRevE.76.036106.

Fuentes de datos y evidencias reproducibles

REEES46. eCommerce behavior data from multi category store. Kaggle, 2019. Los recortes y las simulaciones se describen en README.md. Documentación de NetworkX: https://networkx.org/documentation/.

Los resultados y las corridas individuales se conservan en output/benchmarks/report.json y 1K.json; la evaluación exploratoria, en output/benchmarks_scaling_smoke/report.json; y la ejecución integrada, en output/results/last_pipeline.json. Las pruebas se encuentran en tests/ y el fundamento metodológico, en docs/FUENTES_Y_METODOLOGIA.md.